Local coding-agent stack on Blackwell RTX 5070 / 12 GB

1. Frontends

OpenHands
Docker isolated

OpenCode
Works well with Qwen 3 variants

QwenCode
Claude Code style workflow

Aider / shells / others
Additional frontends sampled lightly

2. Proxy / translation layer

tool_calls_proxy.py
Bridges Qwen XML / JSON into OpenAI tool_calls
Also patches missing required args like description
Runs on port 8100

Optional direct path
OpenHands can speak Qwen 2.5 without this bridge
Still useful for unified behavior

3. Model servers

Ollama / llama.cpp
Best hybrid CPU/GPU behavior
Useful when context spills past GPU
Strong practical fit on 12 GB VRAM
q8_0 KV cache reduces VRAM footprint

vLLM
Finer control and parser plugin path
fp8 KV cache only on Blackwell;
7B model has GQA ratio bug
14B unaffected; viable when model fits
Worth using for tighter throughput control

4. Models

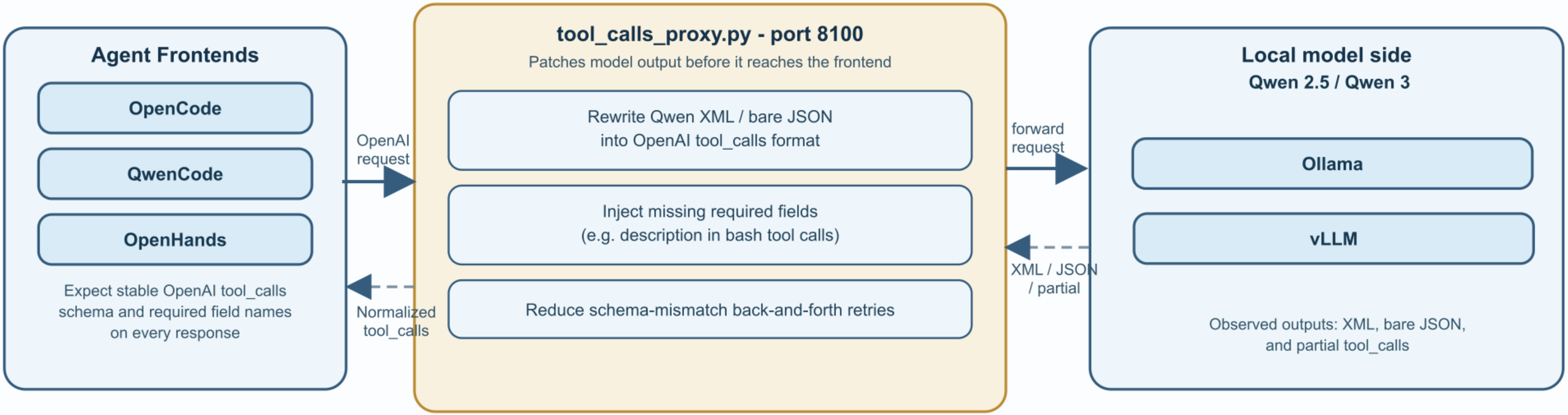
Qwen 2.5
Tool output as XML or bare JSON
Needs proxy for most frontends

Qwen 3 / Qwen 3.5
Native OpenAI tool_calls support
Still had schema quirks in practice

Quantized GGUF builds
Q4_K_M params, Q8_0 KV cache
32K to 64K context target
8B class is the recommendation

Observed protocol mismatch: some frontends want OpenAI tool_calls, while Qwen 2.5 often emits XML or bare JSON. The proxy closes that gap.

Tool-calls proxy — Qwen on a 12 GB Blackwell setup



Why tool_proxy?

Limited 12GB VRAM leaves less room for the model to self-correct schema mismatches. The proxy removes the back and forth retries